

Balotario de Preparación: Lógica y Programación Funcional (Haskell)

Este documento contiene una guía estructurada de 10 ejercicios diseñados para reforzar las competencias clave del bloque de programación funcional en Haskell. Cada ejercicio incluye su descripción, objetivo de aprendizaje y una propuesta de solución formal.

Bloque 1: Guardas y Cláusulas Locales (where)

Objetivo: Comprender la estructuración de flujos lógicos mediante guardas y la optimización de cálculos repetitivos o transformaciones previas utilizando declaraciones locales.

Ejercicio 1: Clasificación de Temperaturas

Implemente la función `clasificarTemperatura` que reciba un valor de tipo `Double`. Debe devolver "Frío" si es menor a 15, "Templado" si está entre 15 y 25 (inclusive), y "Cálido" si es mayor a 25. Es obligatorio el uso de guardas (`|`) para estructurar el control de flujo.

```
clasificarTemperatura :: Double -> String
clasificarTemperatura t
  | t < 15      = "Frío"
  | t <= 25     = "Templado"
  | otherwise  = "Cálido"
```

Ejercicio 2: Evaluación de Rendimiento Escolar

Escriba la función `evaluarNota` que reciba una nota final (`Double`). Debe retornar "Reprobado" si es menor que 10.5, "Regular" de 10.5 a 14, y "Destacado" si es mayor a 14. Utilice obligatoriamente la cláusula `where` para calcular un beneficio hipotético de +1 punto de gracia sobre la nota ingresada antes de evaluarla.

```
evaluarNota :: Double -> String
evaluarNota notaIngresada
  | notaFinal < 10.5  = "Reprobado"
  | notaFinal <= 14.0 = "Regular"
  | otherwise        = "Destacado"
  where
    notaFinal = notaIngresada + 1.0
```

Ejercicio 3: Estado de Inventario de Texto

Cree la función `analizarStock` que reciba una cadena de texto (`String`). Debe devolver "Vacio" si la longitud es 0, "Crítico" si la longitud es menor que 5, y "Óptimo" en cualquier otro caso. Use una cláusula `where` para calcular la longitud de la cadena una sola vez.

```
analizarStock :: String -> String
```

```
analizarStock cadena
  | len == 0    = "Vacío"
  | len < 5     = "Crítico"
  | otherwise   = "Óptimo"
where
  len = length cadena
```

Bloque 2: Manipulación de Listas y Funciones Estándar

Objetivo: Dominar el comportamiento de las funciones primitivas de deconstrucción de estructuras (head, tail) y la construcción mediante el operador de cons (:).

Ejercicio 4: Recorte y Duplicado de Cabecera

Diseñe la función `duplicarResto` que tome una lista de enteros, ignore por completo su primer elemento utilizando `tail` y multiplique por 2 todos los elementos de la sublista restante utilizando listas por comprensión.

```
duplicarResto :: [Int] -> [Int]
duplicarResto lista = [x * 2 | x <- tail lista]
```

Ejercicio 5: Alteración Estricta de Cola

Escriba la función `procesarCola` que reciba una lista de números decimales (`[Double]`). La función debe descartar el primer elemento y sumarle 10.0 únicamente al nuevo primer elemento (la cabeza de la cola resultante).

```
procesarCola :: [Double] -> [Double]
procesarCola lista = (head sublista + 10.0) : tail sublista
  where
    sublista = tail lista
```

Bloque 3: Recursividad Numérica y Casos Base

Objetivo: Diseñar algoritmos iterativos puros controlando las llamadas recursivas y asegurando la correcta convergencia hacia el caso base.

Ejercicio 6: Generador de Pares Descendentes

Cree una función llamada `paresHaciaAbajo` que reciba un número entero positivo `n`. Debe generar una lista con todos los números pares desde `n` (si es par) disminuyendo de 2 en 2 hasta llegar al caso base de 0.

```
paresHaciaAbajo :: Int -> [Int]
paresHaciaAbajo n
```

```

| n <= 0      = [0]
| odd n       = pairsDown (n - 1)
| otherwise = n : pairsDown (n - 2)
where
    pairsDown x
        | x <= 0      = [0]
        | otherwise = x : pairsDown (x - 2)

```

Ejercicio 7: Sumatoria Inversa Progresiva

Escriba la función `listaProgreso` que reciba un entero `n` y construya una lista que sume acumulativamente de tres en tres hacia atrás (`n`, `n-3`, `n-6`...) hasta que el valor sea menor o igual a 1. Defina explícitamente el caso base.

```

listaProgreso :: Int -> [Int]
listaProgreso n
    | n <= 1      = [1]
    | otherwise = n : listaProgreso (n - 3)

```

Bloque 4: Funciones de Orden Superior (filter)

Objetivo: Abstraer el procesamiento y filtrado de colecciones delegando el criterio de selección a un predicado o función booleana interna.

Ejercicio 8: Filtrado de Textos Extensos

Utilice la función nativa `filter` para implementar `filtrarPalabrasLargas`. La función debe recibir una lista de cadenas (`[String]`) y retornar una nueva lista conteniendo únicamente aquellas palabras cuya longitud sea estrictamente mayor a 5 caracteres.

```

filtrarPalabrasLargas :: [String] -> [String]
filtrarPalabrasLargas lista = filter (\p -> length p > 5) lista

```

Ejercicio 9: Aislamiento de Saldos Deudores

Implemente la función `filtrarNegativos` usando `filter`. Debe recibir una lista de montos financieros (`[Double]`) y devolver una lista exclusivamente con los valores menores a 0.0.

```

filtrarNegativos :: [Double] -> [Double]
filtrarNegativos montos = filter (< 0.0) montos

```

Ejercicio 10: Control de Acceso por Rango Impar

Cree la función `filtrarCodigosImpares` que reciba una lista de identificadores numéricos enteros. Utilizando `filter` y la función predefinida `odd`, retorne una estructura limpia con los códigos que cumplan la condición.

```

filtrarCodigosImpares :: [Int] -> [Int]
filtrarCodigosImpares codigos = filter odd codigos

```